```
main()
{
    int x, y;
    Scanf("%d%d", &x, &y);
int res = Sum(x, y);
    printf("%d", res);
}
```

```
int Sum(int x, int y)
{
  return (x+y);
}
```

A function which calls / invokes another function is called calling function.

Called function?

A function which is called by another function

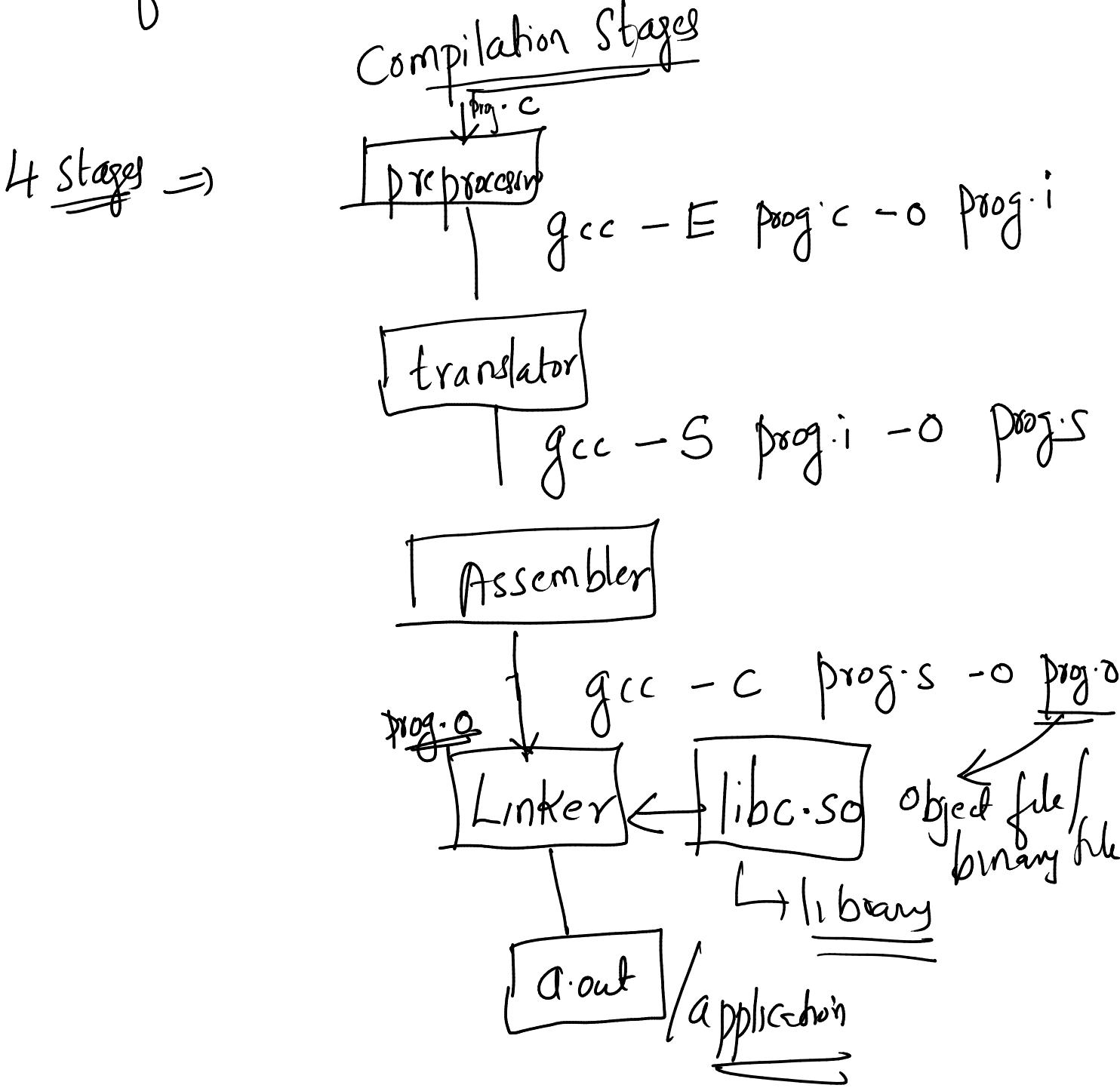
---

1) Code reusability
2) easy to debug
3) Code readability increase

```
main()
{   num = 256
  if(checkprime(num))
  {
    if(checkpalin(dg))
    {
    }
  }
```

```
if(num%10
{
while(num>0)
{
  rem = num%10
  if(rem==5)
  {
  }
}
}
```

reference (address) of called function is known to the Calling function.



4 stages ⇒

Linking

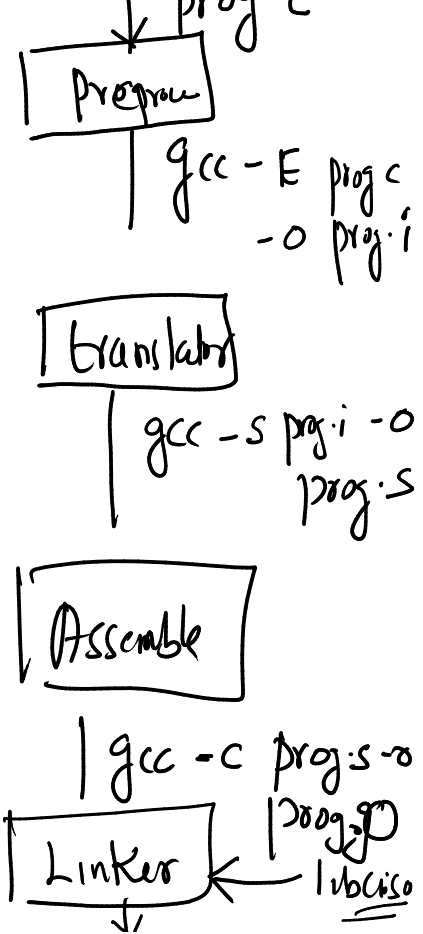
- Static linking
- dynamic linking
  - dynamic load time linking
  - dynamic run time linking

Static Linking

→ Userdefined functions are statically Linked during compile time. ✓

⟹ functions from static library file is statically Linked or copied to the executable.

```
#include<stdio.h>
int sum(int x, int y);
int main()
{
    int x, y;
    scanf("%d%d", &x, &y);
    int r=sum(x,y);
    printf("%d", r);
}
```



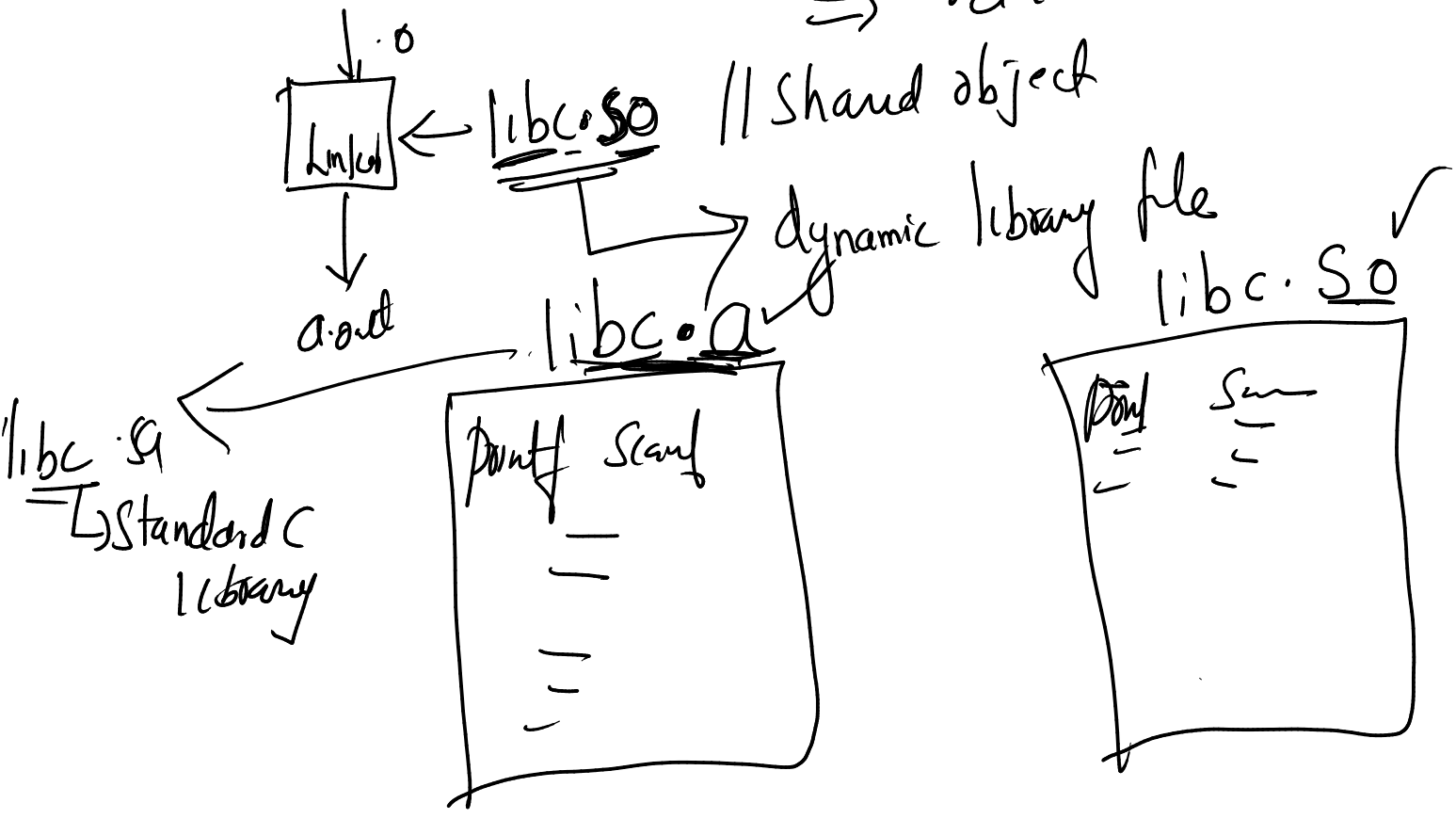
Types of library file

- Static library file
- dynamic library file

1) If linker follows static linking, it make use of static library file
2) If linker follows dynamic linking, it make use of dynamic library file

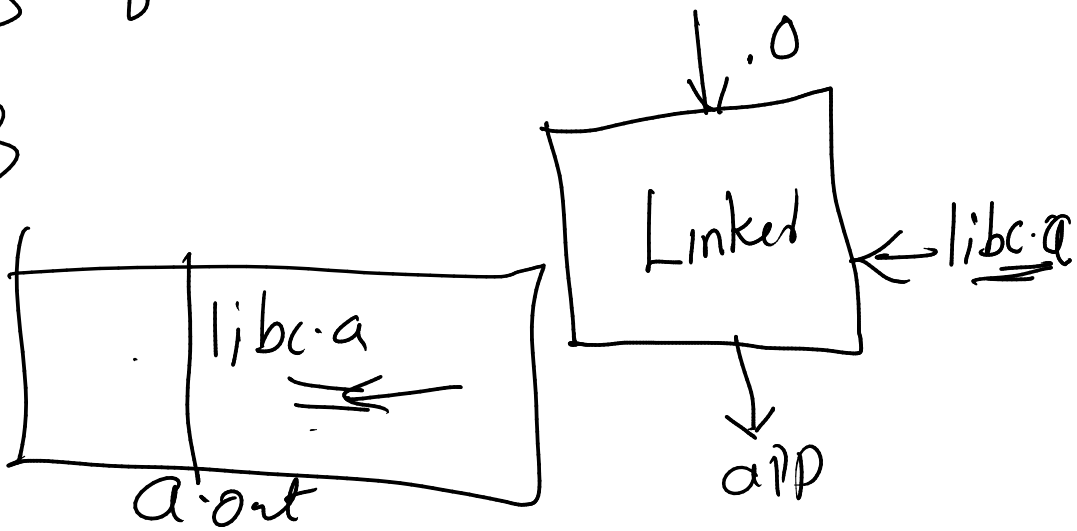
Static library file ⇒ .a → linux
.lib ⇒ windows.

dynamic library file ⇒ ~~s~~ Shared object
⇒ .so in linux
⇒ .dll in windows.



```
main()
{
    int x, y;
    scanf("%d%d", &x, &y);
    int res = sum(x, y);
    printf("%d", res);
}
```

```
sum
```



sqrt(num)

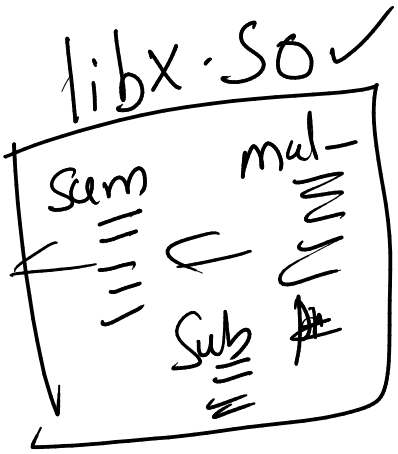
gcc prog.c -lm

libm.so

mathematical lib

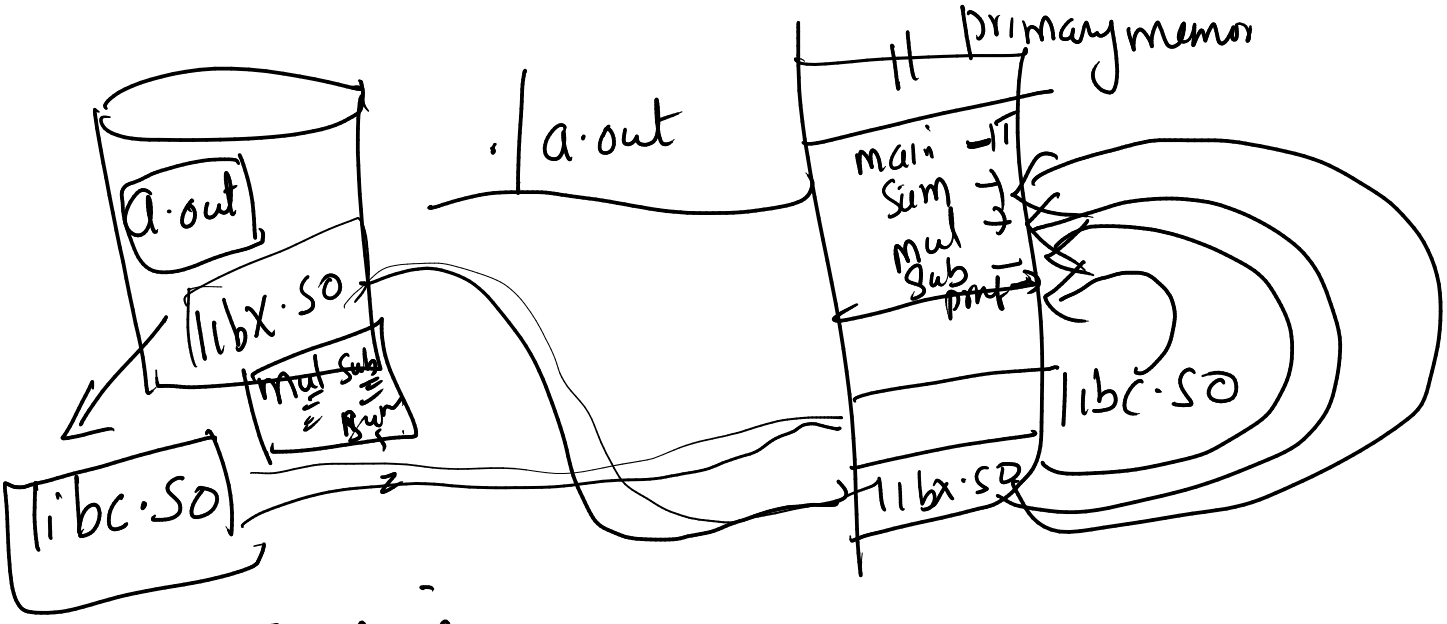
dynamic linking

libx.so



T. main
∪ sum
∪ mul
∪ sub
∪ div

functions from dynamic library file will be linked after the compile time.



T. main

U - sum

U - mul

U - sub

U - prnt